



life.augmented



STM32WL55

Dual core project Interprocessor communication controller

T.O.M.A.S. Team

Goals of the session

Using STM32CubeIDE we will demonstrate how to:

- Select, configure and use interprocessor communication (IPCC) and shared memory concept
- Monitor peripherals within debug session on two cores
- Use built in Single Wire Viewer (SWV) within STM32CubeIDE (with graphical view of monitored variable)

Prerequisites

- NUCLEO-WL55JC2 board
- Micro USB cable
- Single wire to connect two standard gold-pins (raster 2.54mm)
- PC with preinstalled the following software:
 - STM32CubeIDE (in version 1.5.0 or newer)
 - STM32WL Cube library (in version 1.0.0 or newer)
- Theory video about Inter Processor Communication Controller:
<https://youtu.be/3X29gLWUU2U>

Interprocessor communication controller (IPCC) theory



Interprocessor communication (IPCC) in STM32WL55

main points

- Interprocessor communication controller (IPCC) is used for communication between two cores:
 - CPU1, C1 is assigned to ARM CortexM4
 - CPU2, C2 is assigned to ARM CortexM0+
- IPCC contains 12 channels (6 per each direction CPU1->CPU2 and CPU2->CPU1)
- These can work:
 - in simplex mode – data transfer in a single direction from sending to receiving core
 - in half-duplex mode – both cores are exchanging data. After one core sends data, the other one sends a response to it (kind of ping-pong operation)

Interprocessor communication (IPCC) in STM32WL55

main points

- IPCC can generate interrupts on selected (masked) channels activity
 - Once data has been loaded into shared area (ipcc_rx_occupied)
 - Once data has been read from shared area (ipcc_tx_free)
- To exchange data both cores must use the same IPCC channel and the same shared memory buffers
- IPCC doesn't handle buffers nor memories (it is up to the developer to handle them)

Configuration flow using HAL libraries done within code generated by STM32CubeIDE

- Enable the IPCC interface clock
- In case of interrupt mode selection (default within STM32CubeMX and STM32CubeIDE generated code)
 - Configure the IPCC interrupt priority
 - Enable the NVIC IPCC IRQ
- In case of polling mode selection, it is up to the user to poll the status of the communication

Communication flow using HAL libraries

user configuration part

- Select a common IPCC channel (from 0 to 5) for both cores
- Specify the direction of the communication:
 - IPCC_CHANNEL_DIR_TX for the core which would transfer the data
 - IPCC_CHANNEL_DIR_RX for the core which would accept the data
- Choose the communication mode: polling mode or interrupt mode.
- In case of interrupt mode:
 - Implement the interrupt callbacks for transmission and reception
 - Associate those callback to the corresponding channel and direction using HAL_IPCC_ActivateNotification().

Communication flow using HAL libraries polling mode

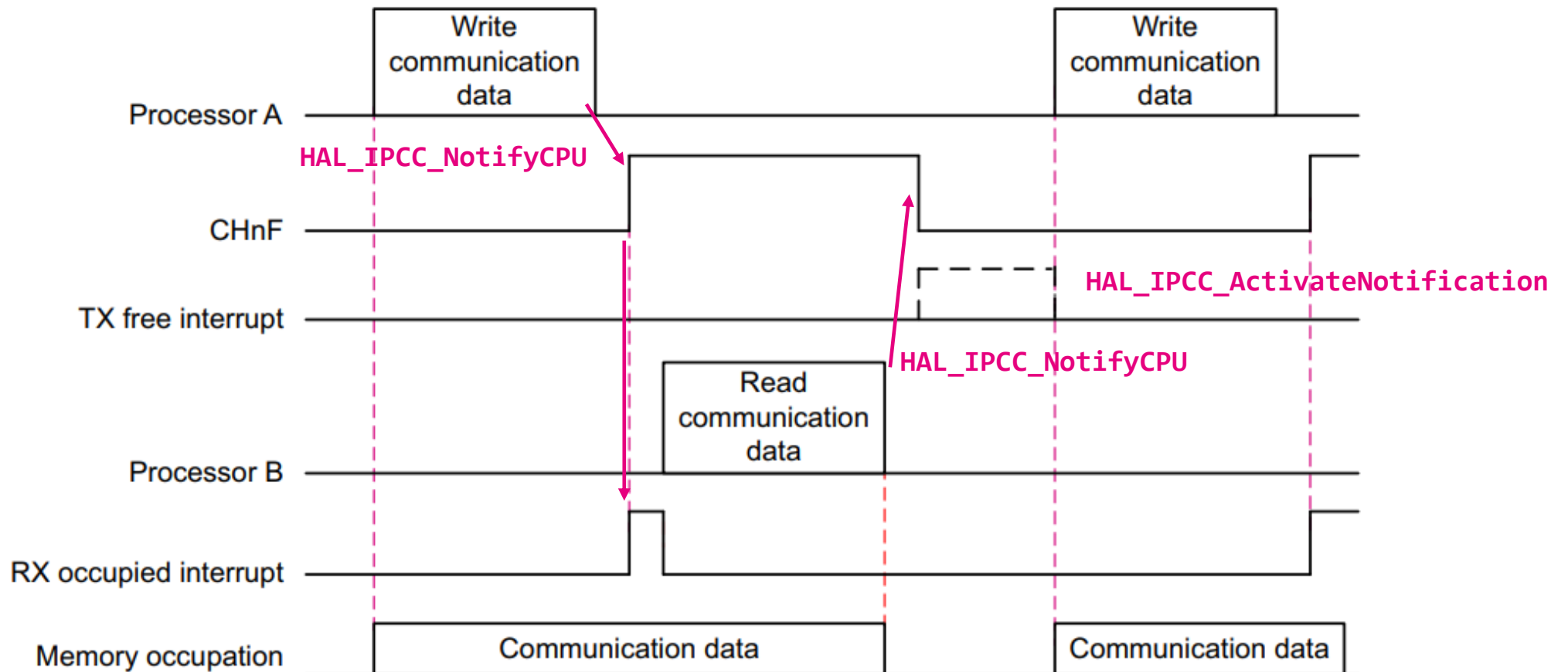
- To send information, use **HAL_IPCC_NotifyCPU()** with IPCC_CHANNEL_DIR_TX.
To know when the other processor has handled the notification, poll the communication using **HAL_IPCC_NotifyCPU()** with IPCC_CHANNEL_DIR_TX.
- To receive information, poll the status of the communication with **HAL_IPCC_GetChannelStatus()** with IPCC_CHANNEL_DIR_RX.
To notify the other processor that the information has been received, use **HAL_IPCC_NotifyCPU()** with IPCC_CHANNEL_DIR_RX.

Communication flow using HAL libraries interrupt mode

- Implement a callback function which would be called at the end of IPCC interrupt
- Assign a callback for the channel and the direction using **HAL_IPCC_ActivateNotification()**. This callback will be triggered under interrupt.
- To transmit information, use **HAL_IPCC_NotifyCPU()** with IPCC_CHANNEL_DIR_TX. The callback configured with **HAL_IPCC_ActivateNotification()** and IPCC_CHANNEL_DIR_TX will be triggered once the communication has been handled by the other processor.
- To receive information, the callback configured with **HAL_IPCC_ActivateNotification()** and IPCC_CHANNEL_DIR_RX will be triggered on reception of a communication. To notify the other processor that the information has been received, use **HAL_IPCC_NotifyCPU()** with IPCC_CHANNEL_DIR_RX.

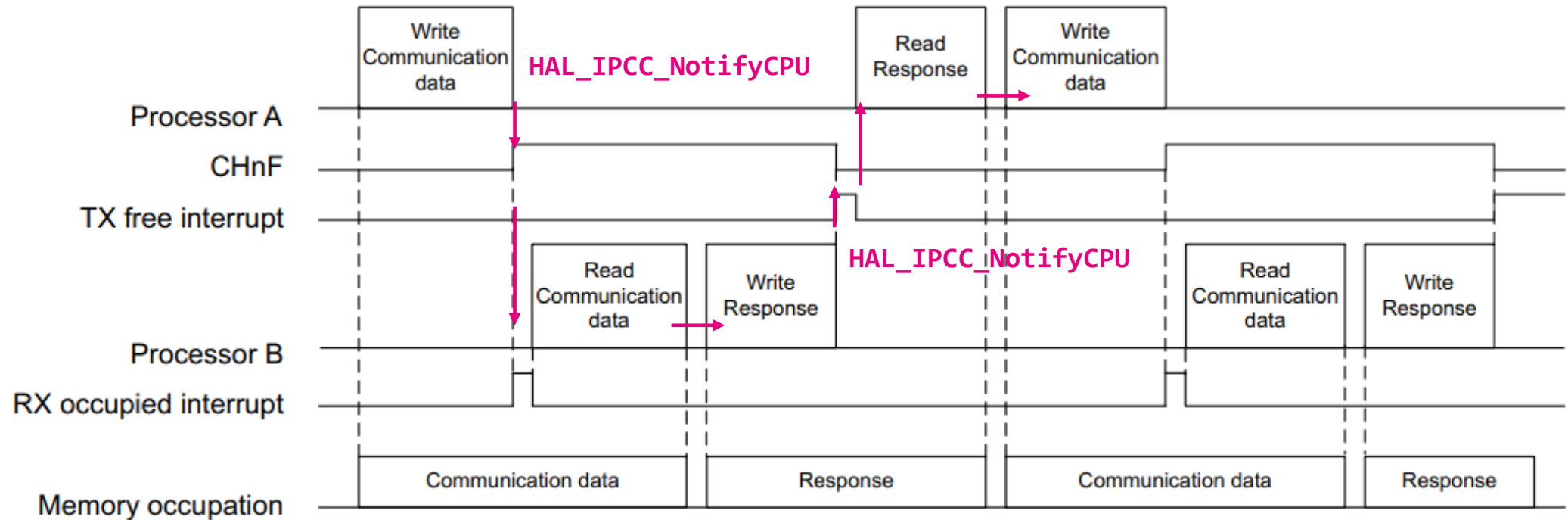
Interprocessor communication (IPCC) in STM32WL55 extract from Reference Manual (RM0453)

- IPCC Simplex



Interprocessor communication (IPCC) in STM32WL55 extract from Reference Manual (RM0453)

- IPCC Half-duplex



Interprocessor communication (IPCC) API 1/4

key functions from stm32wlxx_hal_ipcc.c

- STM32WL HAL functions present in STM32CubeMX and STM32CubeIDE generated code
 - Function executed from main() at initialization stage (main.c files for both cores)
`HAL_StatusTypeDef HAL_IPCC_Init(IPCC_HandleTypeDef *hipcc)`
 - Function executed from HAL_IPCC_Init() to set priority and enable IPCC interrupt (stm32wlxx_hal_msp.c files for both cores)
`void HAL_IPCC_MspInit(IPCC_HandleTypeDef *hipcc)`
 - Functions executed from IPCC interrupt service routines (stm32wlxx_it.c files for both cores)
`void HAL_IPCC_TX_IRQHandler(IPCC_HandleTypeDef *const hipcc)`
`void HAL_IPCC_RX_IRQHandler(IPCC_HandleTypeDef *const hipcc)`

Interprocessor communication (IPCC) API 2/4

key functions from stm32wlxx_hal_ipcc.c

Configuration of selected channel with activation of its notifications (interrupt mode)

```
HAL_StatusTypeDef HAL_IPCC_ActivateNotification
```

```
(
```

```
IPCC_HandleTypeDef *hipcc,
```

```
uint32_t ChannelIndex,
```

```
IPCC_CHANNELDirTypeDef ChannelDir,
```

```
ChannelCb cb
```

```
)
```

IPCC handler

IPCC channel: 0..5

Communication direction:

- **IPCC_CHANNEL_DIR_TX** -> to inform remote core that data has been sent to the shared buffer
- **IPCC_CHANNEL_DIR_RX** -> to inform remote core that data has been read from the shared buffer

Callback function which will be called at the end of interrupt service routine (it will be used instead of the default callbacks implemented in IPCC HAL library module)

Return value:

- **HAL_OK** – operation successful
- **HAL_ERROR** – IPCC busy, not initialized or its handler is not defined

Interprocessor communication (IPCC) API 3/4

key functions from stm32wlxx_hal_ipcc.c

Deactivation of the notifications of selected channel (interrupt mode)

```
HAL_StatusTypeDef HAL_IPCC_DeActivateNotification
```

```
(  
  IPCC_HandleTypeDef *hipcc,  
  uint32_t ChannelIndex,  
  IPCC_CHANNELDirTypeDef ChannelDir  
)
```

IPCC handler

IPCC channel: 0..5

Communication direction:

- **IPCC_CHANNEL_DIR_TX** -> to inform remote core that data has been sent to the shared buffer
- **IPCC_CHANNEL_DIR_RX** -> to inform remote core that data has been read from the shared buffer

Return value:

- **HAL_OK** – operation successful
- **HAL_ERROR** – IPCC busy, not initialized or its handler is not defined

Interprocessor communication (IPCC) API 4/4

key functions from stm32wlxx_hal_ipcc.c

Notification of remote core about operation on shared buffer (write or read)

```
HAL_StatusTypeDef HAL_IPCC_NotifyCPU
```

```
(
```

```
  IPCC_HandleTypeDef const *const hipcc,
```

```
  uint32_t ChannelIndex,
```

```
  IPCC_CHANNELDirTypeDef ChannelDir
```

```
)
```

IPCC handler

IPCC channel: 0..5

Communication direction:

- **IPCC_CHANNEL_DIR_TX** -> to inform remote core that data has been sent to the shared buffer
- **IPCC_CHANNEL_DIR_RX** -> to inform remote core that data has been read from the shared buffer

Return value:

- **HAL_OK** – operation successful
- **HAL_ERROR** – IPCC busy, not initialized or its handler is not defined

NUCLEO-WL55JC2 board preparation

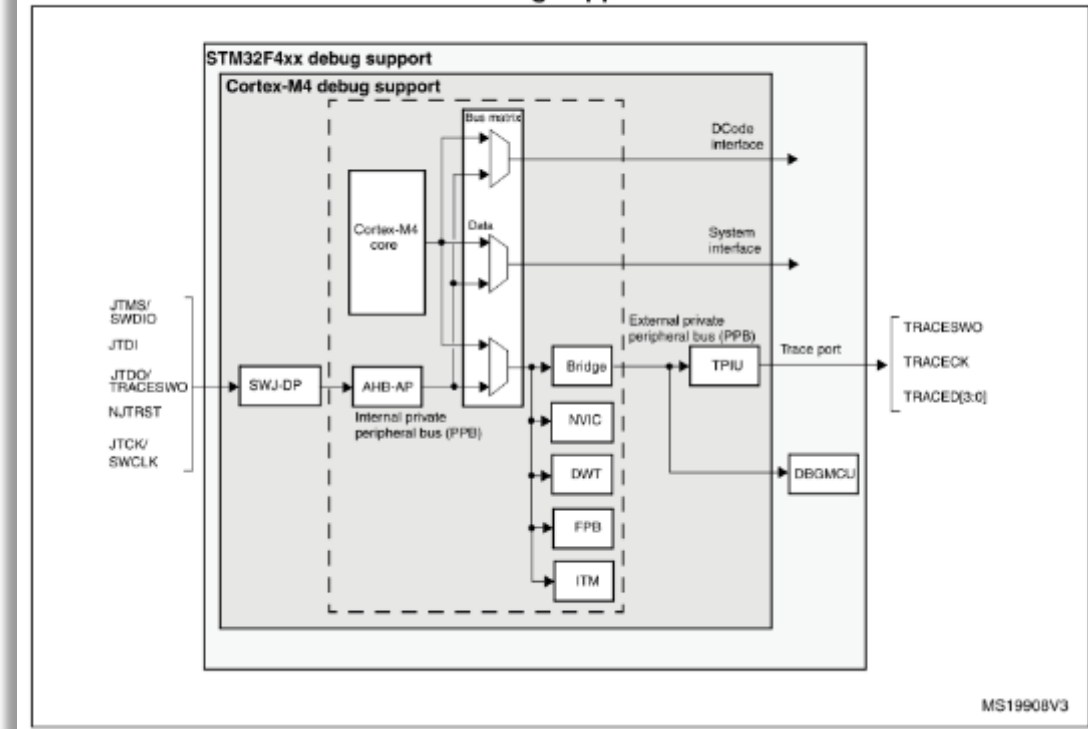


NUCLEO-WL55JC2 board preparation

Introduction

- Most of ARM CortexM cores (except ARM CortexM0 and M0+ cores) there is peripheral called Instrumentation Trace Macrocell (**ITM**) available
- This peripheral can be used to send-out data from MCU over Single Wire Output (SWO) pin
- STM32CubeIDE can display this information during debug (either in terminal-like window or in graphical way)
- On NUCLEO-WL55JC board SWO pin (PB3) is not connected to the STLink by default
- It needs to be either routed by wire or by solder bridges configuration change (see next slides)

Figure 483. Block diagram of STM32 MCU and Cortex®-M4 with FPU-level debug support

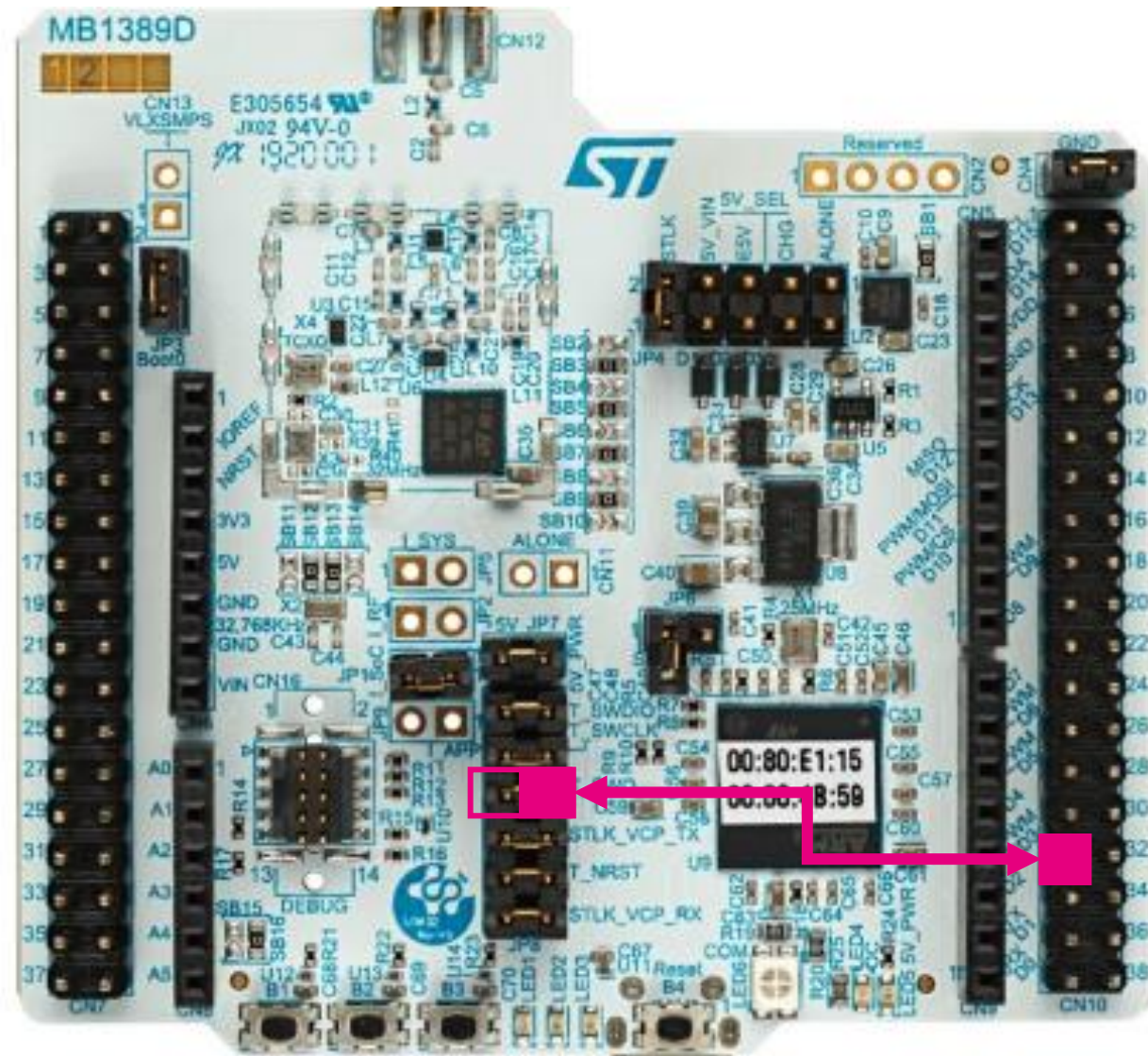
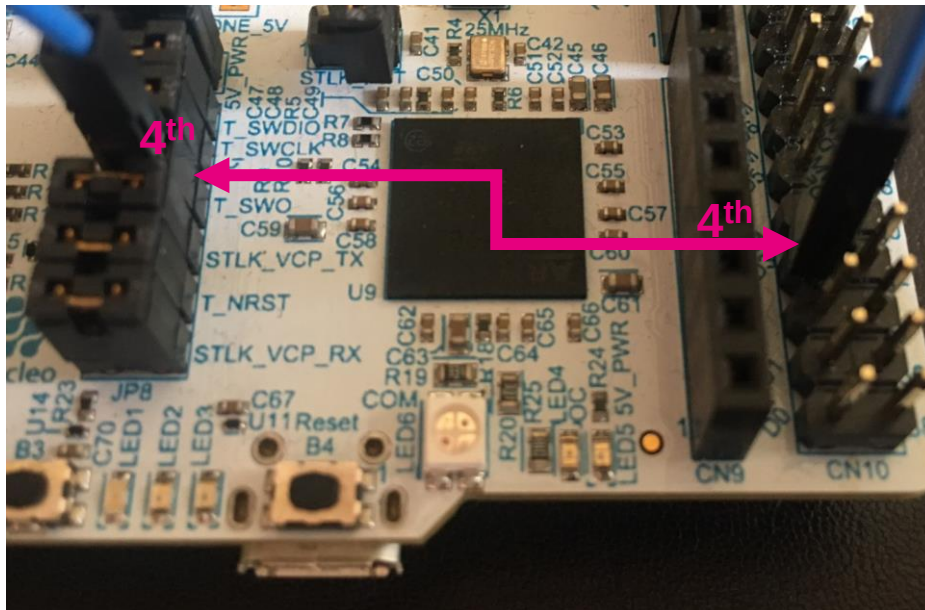


NUCLEO-WL55JC2 board preparation

SWO pin connection – by wire

Routing by wire:

1. please remove jumper **JP8[5-6]** (T_SWO)
2. and connect its right pin to pin **CN10_31**

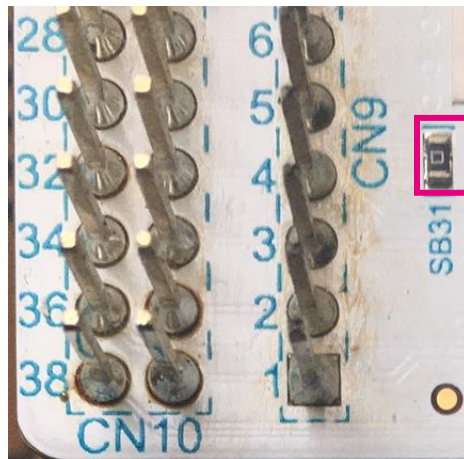
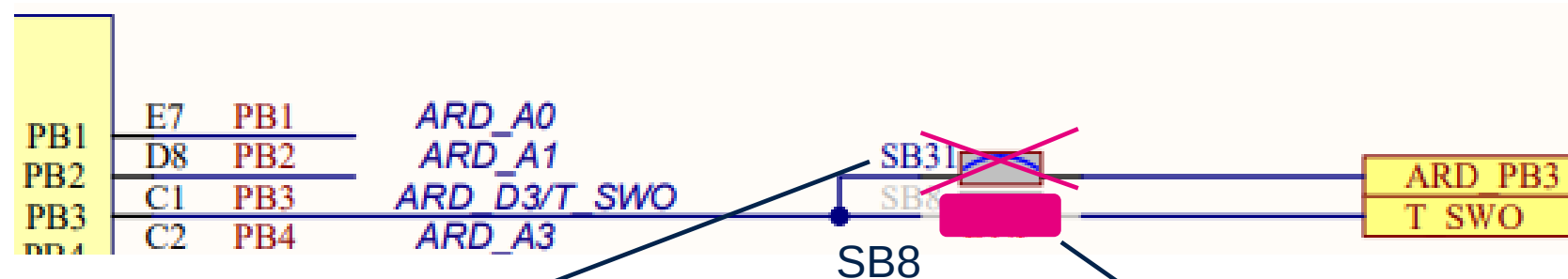


NUCLEO-WL55JC2 board preparation

SWO pin connection – by solder bridges change

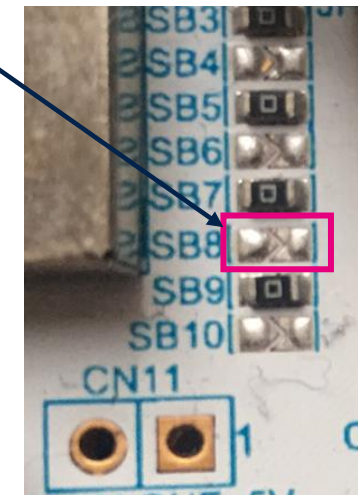
Routing by solder bridges configuration change:

- please remove solder bridge SB31 (bottom side of the board, close to CN9 connector)
- and short solder bridge SB8 (top side of the board near of the down right corner of the metal shielding)



Bottom side of the board

← **Default** configuration
(before modification) →



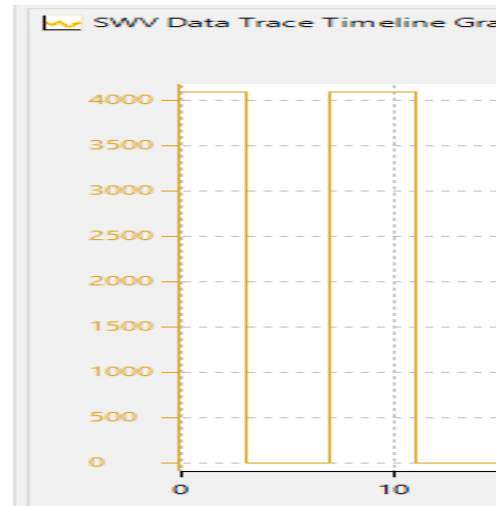
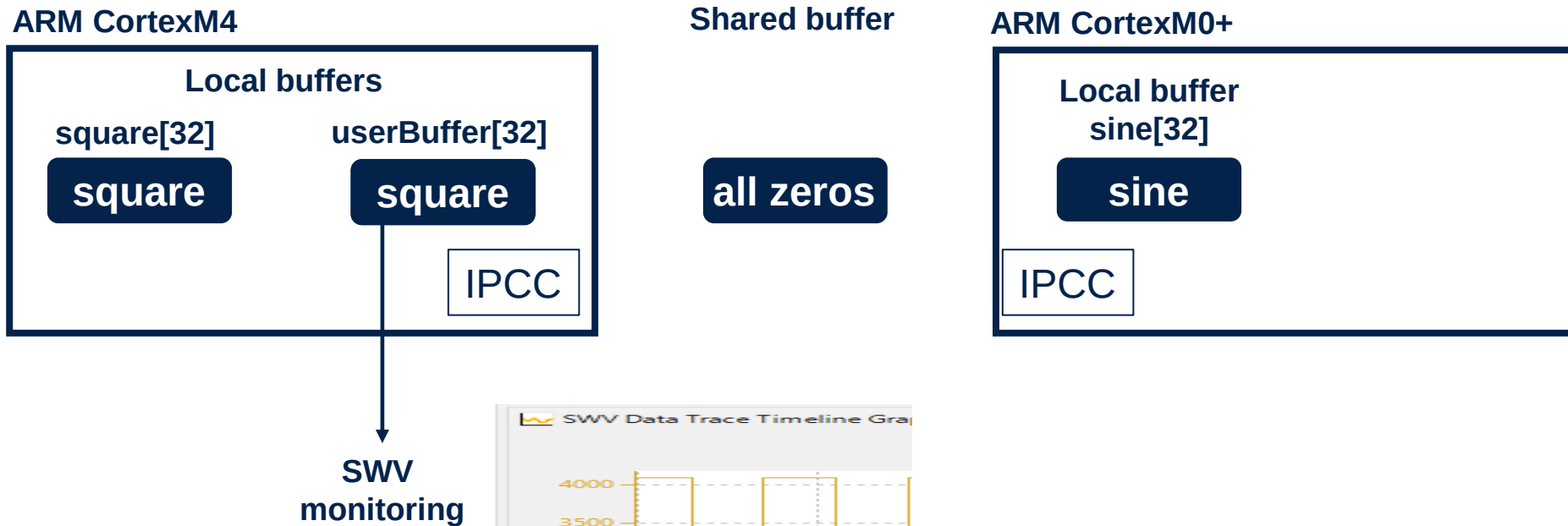
Top side of the board

IPCC example description



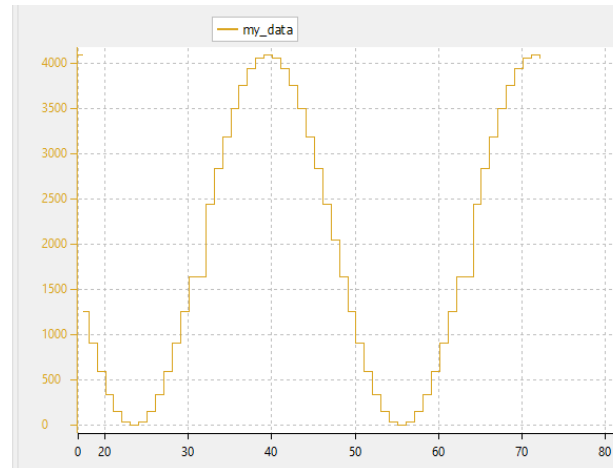
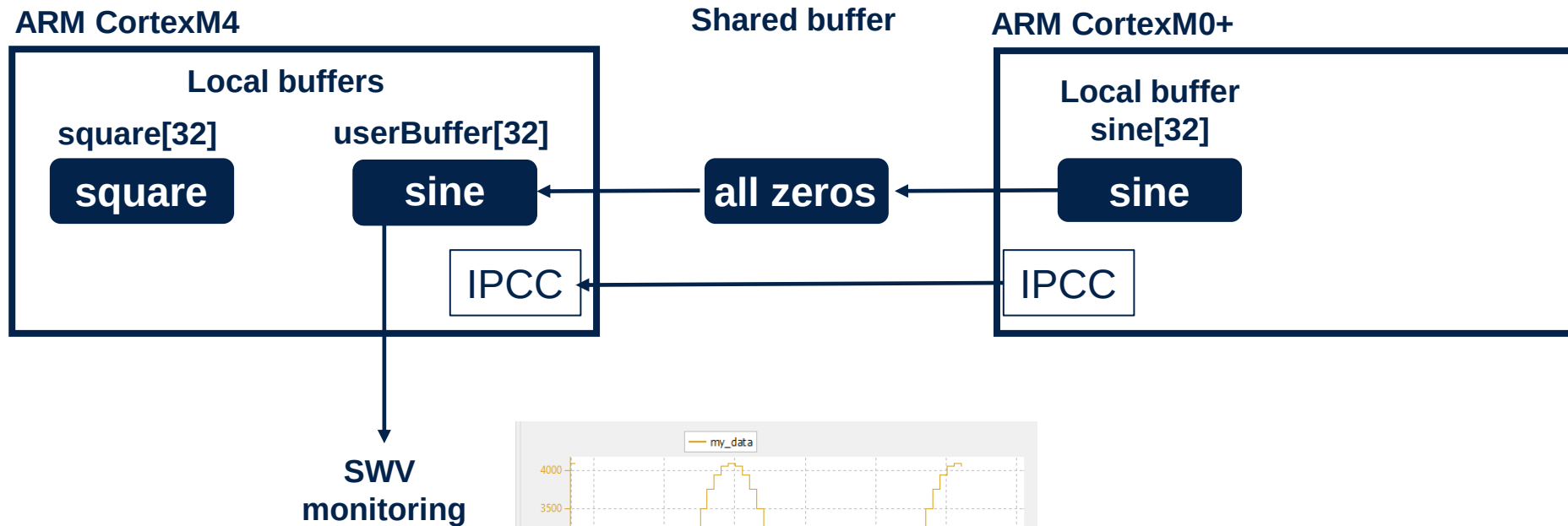
Interprocessor communication controller (IPCC) example description

- After the reset state or B2 button press



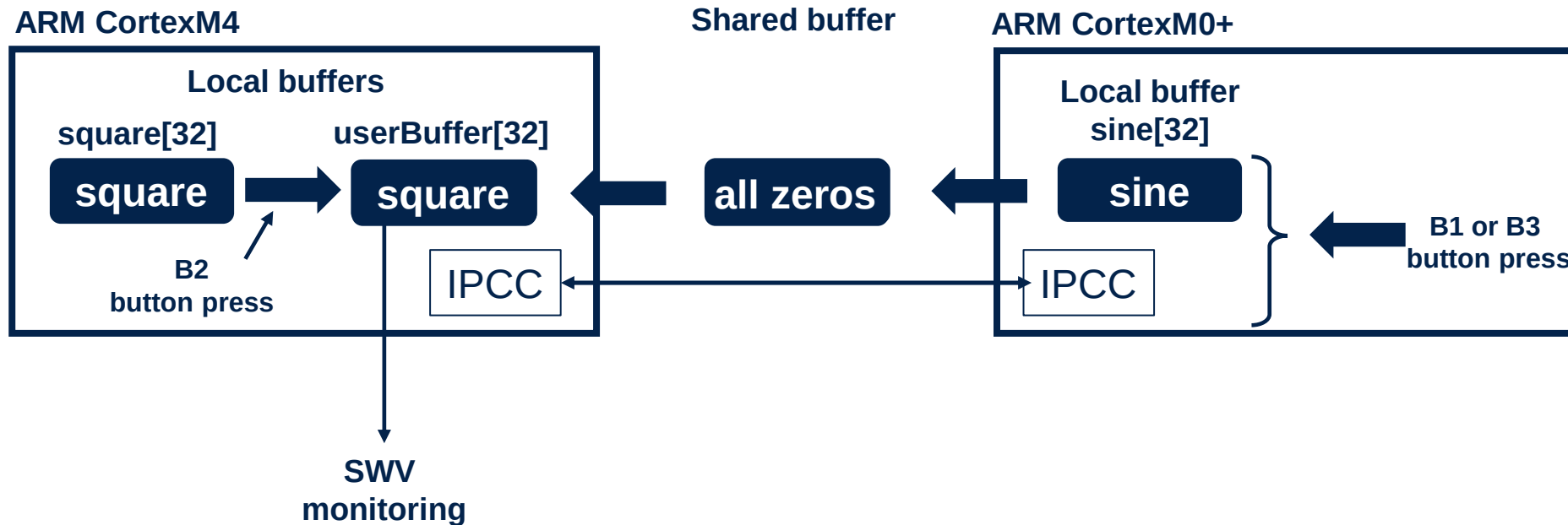
Interprocessor communication controller (IPCC) example description

- After B1 or B3 button press



Interprocessor communication controller (IPCC) example description

- After B2 button press (EXTI IRQs from ARM CortexM4 space)



Interprocessor communication controller (IPCC) example description

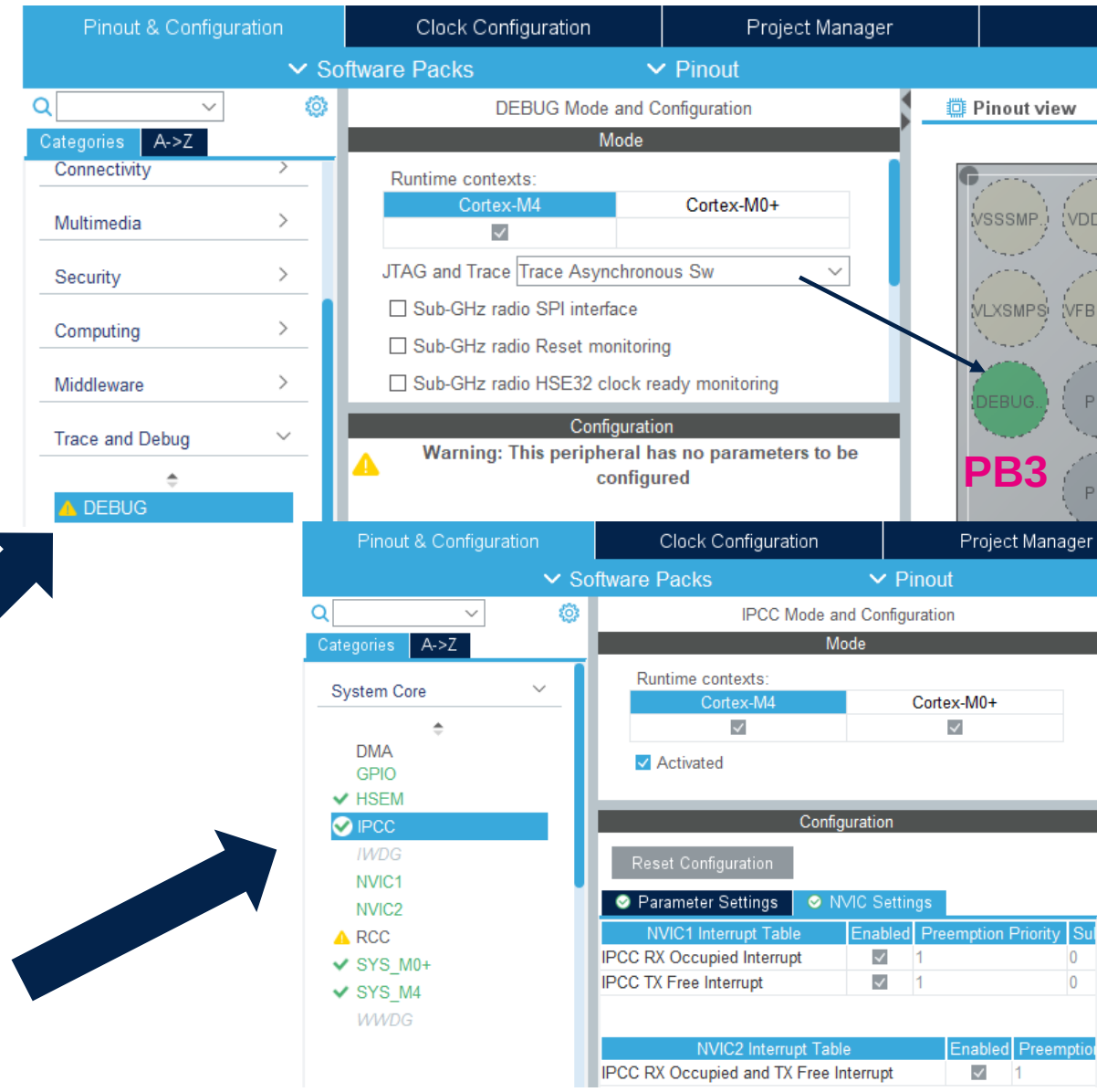
- Create a new dual core project for STM32WL55JC2 MCU and keep default clock configuration(4MHz from MSI)
- Configure PA0, PA1 and PC6 as EXTI lines with pull-ups and set labels (B1, B2 and B3) to them
- Assign PA0 and PC6 to ARM CortexM0+ and PA1 to ARM CortexM4 cores
- Please add IPCC and assign it to both cores with interrupt generation
- Both cores will share a buffer located at the end of SRAM area of the MCU
- After the reset ARM CortexM4 will copy **square[]** local buffer to **workingBuffer[]** and will display its content over Single Wire Viewer (SWV)
- On B1 or B3 button press ARM CortexM0+ will copy **sine[]** local buffer to the **sharedBuffer[]** and inform second core about it via IPCC channel0
- ARM CortexM4 will copy data from **sharedBuffer[]** to its local **workingBuffer[]** (which is displayed over Single Wire Viewer)
- ARM CortexM4 will clear **sharedBuffer[]** and at the end it will inform first core about reception of the data via IPCC channel0.
- On B2 button press ARM CortexM4 will copy **square[]** local buffer to **workingBuffer[]** (like after the reset)

STM32WL55 IPCC project modification



Interprocessor communication (IPCC) example configuration part

- We can reuse previous example (with 3 buttons and 3 LEDs with or without HSEM)
- In case of new project creation it should be done for STM32WL55J MCU with the following peripherals settings:
 - Default clock settings (4MHz, internal clock - for both cores)
 - PA0 as EXTI0 with pull-up assigned to CortexM0+ core
 - PA1 as EXTI1 with pull-up assigned to CortexM4 core
 - PC6 as EXTI6 with pull-up assigned to CortexM0+ core
 - Enable interrupts for EXTI0, EXTI1 and EXTI6 channels in NVIC
- Additionally, we will:
 - select „Trace Asynchronous Sw” within DEBUG.
It will activate additional SWO line PB3 available for ARM CortexM4 core (SWV/SWO is not available for ARM CortexM0 and ARM CortexM0+ cores)
 - enable IPCC for both cores
 - enable IPCC interrupt generation for both cores as well



Interprocessor communication (IPCC) example coding part – linker file tuning

- Locate **sharedBuffer[]** (64 bytes) and **cpu2InitDone** (1 byte) variable in not occupied SRAM memory
- Place it at the end of SRAM area (at 0x2000FFBF and 0x2000FFC0) dedicated to **ARM CortexM0+** (CM0PLUS) core.
- By default it is used by stack, thus mind to modify end of the stack area within linker file **STM32WL55JCIX_FLASH.ld**
 - To do this please decrease **_estack** value by the size of the buffer and common variable, i.e:

```
/* Highest address of the user mode stack */  
_estack = ORIGIN(RAM) + LENGTH(RAM) - 65; /* end of "RAM" Ram type memory */
```

Interprocessor communication (IPCC) example coding part – ARM CortexM4 main.c file

- Add header of the library which contains memset() and memcpy() functions:

```
/* USER CODE BEGIN Includes */  
#include <string.h>
```

- Add some definitions

```
/* USER CODE BEGIN PD */  
#define CPU2_NOT_STARTED      0xFF  
#define CPU2_STARTED         0xA5  
#define BUFFER_SIZE          32
```

Interprocessor communication (IPCC) example coding part – ARM CortexM4 main.c file

- Add declarations of some global variables

```
/* USER CODE BEGIN PV */
__IO uint16_t *sharedBuffer = (uint16_t *)0x2000FFC0;
__IO char *cpu2InitDone = (char *)0x2000FFBF;
__IO uint32_t Receive_flag = 0;
uint16_t userBuffer[BUFFER_SIZE];
uint16_t square[BUFFER_SIZE] = {4095, 4095, 4095, 4095, 0, 0, 0, 0, 4095, 4095, 4095,
                                4095, 0, 0, 0, 0, 4095, 4095, 4095, 4095, 0, 0, 0, 0,
                                4095, 4095, 4095, 4095, 0, 0, 0, 0};

uint16_t my_data=0;
uint32_t ChannelId = 0; /* Any value between 0 and (IPCC_CHANNEL_NUMBER-1) */
uint8_t transfer_trigger = 0;
char i=0;
```

Interprocessor communication (IPCC) example coding part – ARM CortexM4 main.c file

- Add declaration of the function called on new data reception on IPCC channel

```
/* USER CODE BEGIN PFP */  
void Receive_callback(struct __IPCC_HandleTypeDef *hipcc, uint32_t ChannelIndex,  
IPCC_CHANNELDirTypeDef ChannelDir);
```

- Define the function called on new data reception on IPCC channel

```
/* USER CODE BEGIN 4 */  
void Receive_callback(struct __IPCC_HandleTypeDef *hipcc, uint32_t ChannelIndex,  
IPCC_CHANNELDirTypeDef ChannelDir)  
{  
    Receive_flag = 1;  
}
```

Interprocessor communication (IPCC) example coding part – ARM CortexM4 main.c file

- Update existing EXTI interrupt callback

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if(GPIO_PIN_1 == GPIO_Pin)
    {
        transfer_trigger = 1; //flag to trigger transfer from square[] to userBuffer[]
    }
}/* USER CODE END 4 */
```

Interprocessor communication (IPCC) example coding part – ARM CortexM4 main.c file

- At the beginning of main() specify remote CPU (ARM CortexM0+) state as not started

```
/* USER CODE BEGIN 1 */  
*cpu2InitDone = CPU2_NOT_STARTED;
```

- Load square wave data as initial ones to userBuffer[]

```
/* USER CODE BEGIN SysInit */  
memcpy ((uint16_t*)userBuffer, (uint16_t*)square, sizeof(uint16_t)*BUFFER_SIZE);
```

- Activate notification on reception events within selected IPCC channel and assign them to Receive_callback() function

```
/* USER CODE BEGIN 2 */  
if(HAL_IPCC_ActivateNotification(&hipcc, ChannelId, IPCC_CHANNEL_DIR_RX, Receive_callback)  
!=HAL_OK)  
{  
    Error_Handler();  
}
```

Interprocessor communication (IPCC) example coding part – ARM CortexM4 main.c file

- Wait until CPU2 is not started then continue

```
/* USER CODE BEGIN WHILE */
```

```
/* Wait until cpu2 is ready to receive commands */
```

```
while (*cpu2InitDone != CPU2_STARTED);
```

- Just below, within main while(1) loop copy all components of userBuffer[] in the loop into **my_data** variable (to be monitored in debug session)

```
while (1)
```

```
{
```

```
    if(++i < BUFFER_SIZE)
```

```
    {
```

```
        my_data = userBuffer[i]; //this variable will be monitored by SWV
```

```
    }
```

```
    else
```

```
        i = 0;
```

Interprocessor communication (IPCC) example coding part – ARM CortexM4 main.c file

- Just below, within the same main while(1) loop copy data (sent by CM0+) from sharedBuffer[] to userBuffer[] and clear sharedBuffer[] as a reply to CM0+

```
if(Receive_flag == 1)
{
    /* Write communication data to memory */
    memcpy ((uint16_t*)userBuffer, (uint16_t*)sharedBuffer, sizeof(uint16_t)*BUFFER_SIZE);
    /* for Half duplex activities we will erase sharedBuffer as a reply to CM0+*/
    memset((uint16_t*)sharedBuffer, '\0', sizeof(uint16_t)*BUFFER_SIZE);
    /* notify CM0+ core that it can process next data into sharedBuffer[] */
    if(HAL_IPCC_NotifyCPU(&hipcc, ChannelId, IPCC_CHANNEL_DIR_RX) != HAL_OK)
    {
        Error_Handler();
    }

    Receive_flag = 0; //clear the flag raised in IPCC interrupt routine
}
```

Interprocessor communication (IPCC) example coding part – ARM CortexM4 main.c file

- Just below within main while(1) loop in case of EXTI interrupt flag occurrence (transfer_trigger) copy content of square[] buffer to userBuffer[] and clear the flag. Afterwards to slow down the process insert 1 second delay.

```
if(transfer_trigger == 1)
{
    memcpy ((uint16_t*)userBuffer, (uint16_t*)square, sizeof(uint16_t)*BUFFER_SIZE);
    transfer_trigger = 0;
}
```

```
HAL_Delay(1000);
```

```
/* USER CODE END WHILE */
```

Interprocessor communication (IPCC) example coding part – ARM CortexM0+ main.c file

- Add header for the library containing **memset()** and **memcpy()** functions:

```
/* USER CODE BEGIN Includes */  
#include <string.h>
```

- Declare some definitions

```
/* USER CODE BEGIN PD */  
#define CPU2_NOT_STARTED      0xFF  
#define CPU2_STARTED         0xA5  
#define BUFFER_SIZE          32
```

Interprocessor communication (IPCC) example coding part – ARM CortexM0+ main.c file

- Declare some global variables

```
/* USER CODE BEGIN PV */
__IO uint16_t *sharedBuffer = (uint16_t *)0x2000FFC0;
__IO char *cpu2InitDone = (char *)0x2000FFBF;
__IO uint32_t Transmit_flag = 0;
uint16_t userBuffer[BUFFER_SIZE];
uint16_t sine[BUFFER_SIZE] = {2047, 2447, 2831, 3185, 3498, 3750, 3939, 4056, 4095, 4056,
                              3939, 3750, 3495, 3185, 2831, 2447, 2047, 1647, 1263, 909,
                              599, 344, 155, 38, 0, 38, 155, 344, 599, 909, 1263, 1647};
uint32_t ChannelId = 0; /* Any value between 0 and (IPCC_CHANNEL_NUMBER-1) */
uint8_t transfer_trigger = 0;
```

Interprocessor communication (IPCC) example coding part – ARM CortexM0+ main.c file

- Add declaration of the function called on new data transmission over IPCC channel

```
/* USER CODE BEGIN PFP */  
void Transmit_callback(struct __IPCC_HandleTypeDef *hipcc, uint32_t ChannelIndex,  
IPCC_CHANNELDirTypeDef ChannelDir);
```

- Implement definition of the function called on new data transmission over IPCC channel

```
/* USER CODE BEGIN 4 */  
void Transmit_callback(struct __IPCC_HandleTypeDef *hipcc, uint32_t ChannelIndex,  
IPCC_CHANNELDirTypeDef ChannelDir)  
{  
    Transmit_flag = 1;  
}
```

Interprocessor communication (IPCC) example coding part – ARM CortexM0+ main.c file

- Within existing EXTI interrupt callback set transfer_trigger variable to 1

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if(GPIO_PIN_0 == GPIO_Pin)
    {
        transfer_trigger = 1;
    }
    else if (GPIO_PIN_6 == GPIO_Pin)
    {
        transfer_trigger = 1;
    }
}
/* USER CODE END 4 */
```

Interprocessor communication (IPCC) example coding part – ARM CortexM0+ main.c file

- At the beginning of main() clear shared buffer

```
/* USER CODE BEGIN 1 */  
memset((uint16_t*)sharedBuffer, '\0', sizeof(uint16_t)*BUFFER_SIZE);
```

- Notify CM4 core that CM0+ core has started, activate notifications on transmit events within selected IPCC channel and assign them to Transmit_callback() function

```
/* USER CODE BEGIN 2 */  
*cpu2InitDone = CPU2_STARTED;  
  
if( HAL_IPCC_ActivateNotification(&hipcc, ChannelId, IPCC_CHANNEL_DIR_TX, Transmit_callback)  
    != HAL_OK)  
{  
    Error_Handler();  
}
```

Interprocessor communication (IPCC) example coding part – ARM CortexM0+ main.c file

- In case of transfer_trigger flag set, at the beginning of main while(1) loop copy sine[] buffer to shared buffer[], then send notification over IPCC

```
if(transfer_trigger == 1)
{
    /* copy sinewave to shared buffer */
    memcpy ((uint16_t*)sharedBuffer, (uint16_t*)sine, sizeof(uint16_t)*BUFFER_SIZE);

    /* Information to remote CPU that data has been transferred */
    if(HAL_IPCC_NotifyCPU(&hipcc, ChannelId, IPCC_CHANNEL_DIR_TX) != HAL_OK)
    {
        Error_Handler();
    }

    transfer_trigger = 0;
}
```

Interprocessor communication (IPCC) example coding part – ARM CortexM0+ main.c file

- Just below within main while(1) loop just clear Transmit_flag. From this moment we can upload new data to sharedBuffer[] as previous data has been received by CM4 core

```
if(Transmit_flag == 1)
{
    Transmit_flag = 0;
}

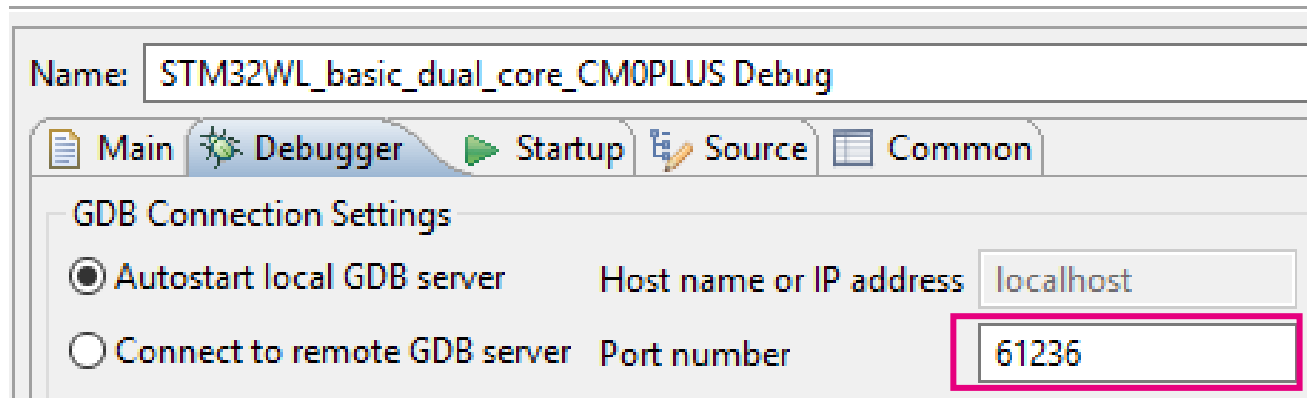
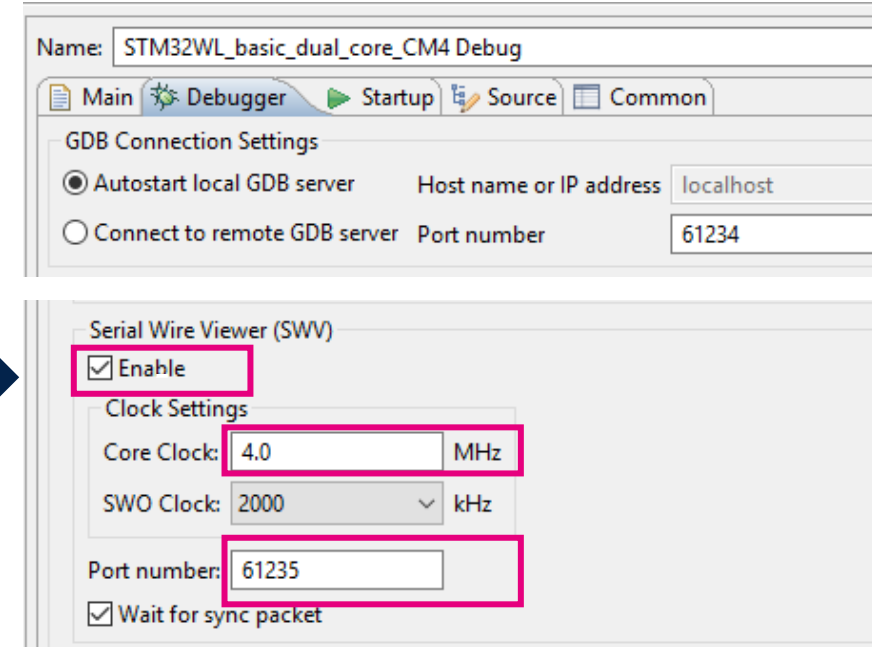
HAL_Delay(500);
/* USER CODE END WHILE */
```

STM32WL55 IPCC debug session



Interprocessor communication (IPCC) example configure debug session – enabling SWV

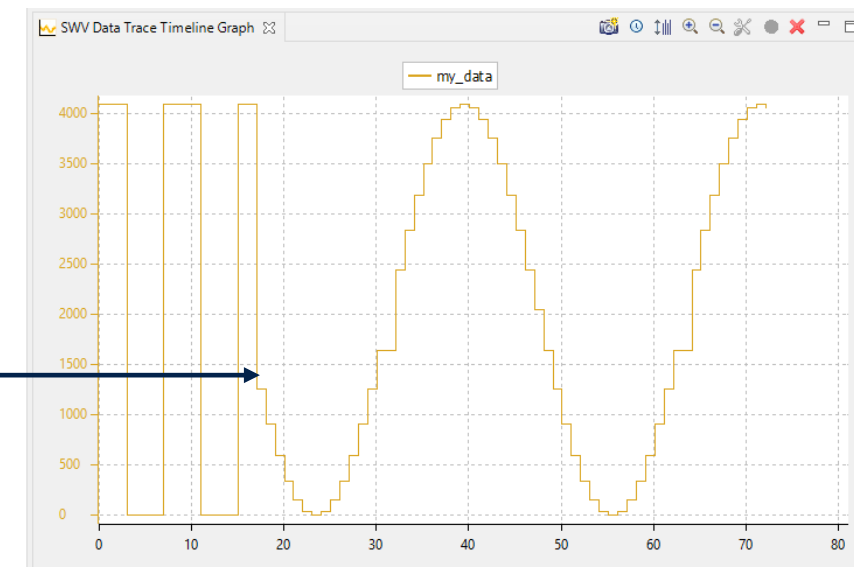
- Within ARM CortexM4 debug configuration we need to:
 - **enable** Single Wire Viewer (SWV),
 - **set Core Clock** to be used in the application (in our case **4MHz**)
 - **select Port number**
By default it will be SWD port number +1 → in this example it is **61235**.
- Within ARM CortexM0+ debug configuration we need to define SWD port number (i.e. to **61236**)



Interprocessor communication (IPCC) example start debug session

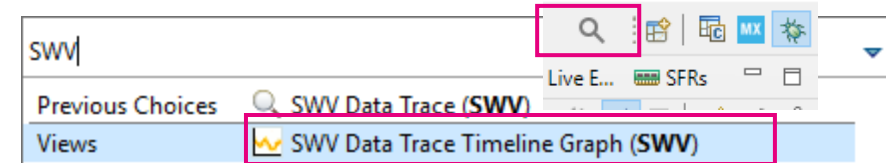
- Start CortexM4 debug configuration, then resume code execution (to start CortexM0+)
- Start CortexM0P debug configuration
- Suspend code execution on both cores
- Configure and open SWV Data Trace window within STM32CubeIDE (see next slide)
- Start trace
- Resume code execution on both cores (Ctrl + selection + resume button)
- Let it run for a while, then press B1 or B3 button on the board
- An example of the result is visible on the right

B1 or B3 button press -> changing square to sine
data within userBuffer

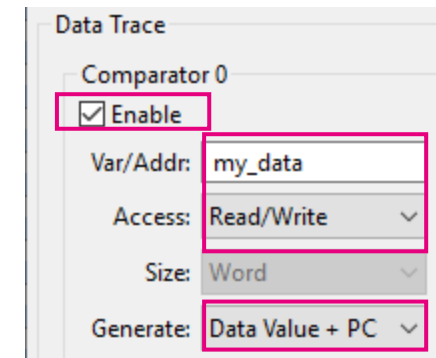
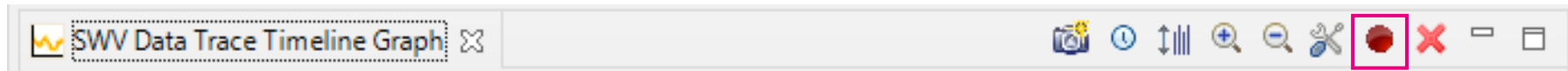


Configuring SWV within STM32CubeIDE

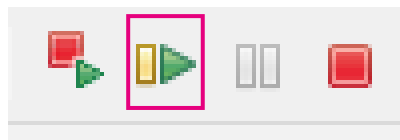
- Suspend code execution on ARM CortexM4 core
- Within debug session click on Quick search icon and type „SWV”
- Select SWV Data Trace Timeline Graph from the list
- Select Configuration
- Enable one of the comparators, insert „my_data” variable, select Read/Write access and generate Data Value + PC. Then press OK.



- Start tracing of the selected variable



- Resume code execution of ARM CortexM4 core



Further information

- IPCC description (based on STM32WB MCUs):

<https://youtu.be/3X29gLWUU2U>

- Mailbox Multiplexer (MBMUX) software layer:

- https://www.st.com/content/ccc/resource/training/technical/product_training/group1/f6/a5/26/cf/3f/dc/46/de/STM32WL-System-Mailbox_Multiplexer_MBXMUX/files/STM32WL-System-Mailbox_Multiplexer_MBXMUX.pdf/jcr:content/translations/en.STM32WL-System-Mailbox_Multiplexer_MBXMUX.pdf

Thank you

© STMicroelectronics - All rights reserved.

ST logo is a trademark or a registered trademark of STMicroelectronics International NV or its affiliates in the EU and/or other countries.

For additional information about ST trademarks, please refer to www.st.com/trademarks.

All other product or service names are the property of their respective owners.



life.augmented